DOCUMENTACIÓN TÉCNICA & ARQUITECTURA

Portafolio Web, Laboratorio Físico & Catálogo Arcade PWA

Autor: Leonardo Viscaya · Stack: Python (Flask), SQLite3, HTML5 Canvas, CSS3 Botánico, JavaScript ES6 & Service Workers PWA

1. Resumen General del Proyecto

El proyecto **personal_web** es una plataforma web integral desarrollada en **Python con Flask** y base de datos **SQLite**, diseñada bajo una estética editorial botánica sin publicidad. Integra tres pilares principales en una sola arquitectura:

- **Portafolio Profesional & Casos de Estudio:** Presentación interactiva con acordeón desplegable de proyectos reales (Plataforma & Estudio Virtual, Bike Fit Marketplace con Stripe Connect y Sistema Interactivo de Asignación), cursor magnético y formulario de contacto conectado a SQLite.
- **Catálogo Arcade Offline (6 Cartuchos PWA):** Colección de 6 juegos clásicos programados desde cero (Buscaminas Pro, Sudoku, Solitario Klondike, Tetris, 2048 y Snake). Cada cartucho genera dinámicamente su propio `manifest.json`, `sw.js` e icono vectorial `icon.svg` para instalarse como aplicación independiente en dispositivos móviles o escritorio.
- **Laboratorio de Física Computacional (Sandbox Matemático):** Simulaciones interactivas en tiempo real sobre HTML5 Canvas (Campo Gravitacional Central, Interferencia de Ondas Armónicas y Campo Vectorial Magnético Dipolar).

2. Estructura de Archivos del Proyecto

```
personal_web/
├── app.py                      # Servidor principal Flask, rutas, API SQLite y generador PWA
├── .env                        # Variables de entorno (puerto, modo debug, rutas de base de
datos y deploy)
├── .env_venv/                  # Entorno virtual de Python con dependencias instaladas
├── requirements.txt            # Dependencias del servidor (Flask, python-dotenv)
├── portfolio.db                # Base de datos relacional SQLite (tablas: messages,
game_scores)
├── documentacion_proyecto.pdf  # Documentación técnica completa del proyecto
├── static/
│   ├── css/
│   │   └── style.css           # Sistema de diseño botánico, modo claro/oscuro, grillas y
animaciones
│   ├── js/
│   │   ├── main.js             # Lógica global: navegación, tema, acordeón, experimentos y
contacto
│   │   └── arcade_engines.js   # Motores completos de los 6 juegos del Arcade +
sincronización SQL
│   ├── manifest.json           # Manifiesto PWA general del sitio
│   └── sw.js                   # Service Worker general con estrategia Network-First
└── templates/
```

```
    ├── index.html              # Portada principal (Hero, Proyectos, Demo Buscaminas,
Sandbox, Contacto)
    ├── arcade.html             # Catálogo visual de los 6 cartuchos instalables
    ├── experimentos.html       # Laboratorio dedicado con las 3 simulaciones físicas
interactivas
    └── game_pwa.html           # Vista individual centrada (Center-Stage) para cada juego del
Arcade
```

3. Arquitectura Backend (`app.py`) y Base de Datos SQLite

3.1. Esquema de Base de Datos (`portfolio.db`)

Al iniciar la aplicación, la función `init_db()` crea automáticamente dos tablas si no existen:

Tabla	Columnas	Propósito
`messages`	`id` (PK), `name` (TEXT), `email` (TEXT), `message` (TEXT), `created_at` (TIMESTAMP)	Almacena los mensajes enviados desde el formulario de contacto de la portada.
`game_scores`	`game_id` (PK TEXT), `best_score` (INTEGER), `updated_at` (TIMESTAMP)	Guarda el récord histórico máximo de cada uno de los 6 juegos del catálogo Arcade.

3.2. Rutas y Endpoints del Servidor

Ruta / Endpoint	Método	Descripción Funcional
`/`	GET	Renderiza `templates/index.html` (Portada principal del portafolio).
`/arcade`	GET	Renderiza `templates/arcade.html` inyectando el diccionario `GAMES`.
`/experimentos`	GET	Renderiza `templates/experimentos.html` (Laboratorio de física computacional).
`/api/contact`	POST	Valida y guarda un nuevo mensaje de contacto en la tabla `messages` de SQLite.
`/api/scores/<game_id>`	GET / POST	Consulta o actualiza el récord histórico del juego si el puntaje enviado supera al actual.
`/arcade/<game_id>/`	GET	Renderiza `templates/game_pwa.html` configurado para el juego solicitado.
`/arcade/<game_id>/manifest.json`	GET	Genera dinámicamente el manifiesto PWA con alcance aislado (`scope: /arcade/<id>/`).
`/arcade/<game_id>/sw.js`	GET	Genera el Service Worker exclusivo del juego para soporte offline independiente.
`/arcade/<game_id>/icon.svg`	GET	Genera el ícono vectorial SVG de 512x512 personalizado para cada cartucho.
`/update_server`	POST	Webhook para despliegue continuo en PythonAnywhere (`git fetch` + `reset --hard`).

4. Arquitectura Frontend y Módulos JavaScript

4.1. Sistema de Diseño CSS (`static/css/style.css`)

El diseño emplea una paleta botánica definida mediante variables CSS en `:root`: `--dust-grey (#dad7cd)`, `--dry-sage (#a3b18a)`, `--fern (#588157)`, `--hunter-green (#3a5a40)` y `--pine-teal (#344e41)`. Cuando el atributo `data-mode="dark"` se activa en la etiqueta `<html>`, las variables de fondo y texto se invierten suavemente.

4.2. Módulos de `static/js/main.js`

- **Módulo 0 (Activos SVG):** Constantes vectoriales para banderas, pico de excavación y minas.
- **Módulo 1 (Cursor Magnético):** Controla `#custom-cursor` y la píldora flotante `#floating-preview`.
- **Módulo 2 (Acordeón de Proyectos):** Alterna la clase `.open` al hacer clic en cada fila `.work-row`.
- **Módulo 3 (Navegación y Tema):** Hace scroll suave hacia las secciones con `data-target="#id"` y alterna el modo claro/oscuro con persistencia en `localStorage`.
- **Módulo 4 (Buscaminas Demo en Portada):** Tablero 8x8 en `index.html` con primer clic seguro y despeje rápido (chording). Se ejecuta únicamente en la portada para no interferir con `arcade_engines.js`.
- **Módulo 5 (Sandbox Físico en Portada):** Controla `#experiment-canvas` permitiendo alternar entre Campo Gravitacional Central e Interferencia de Ondas Armónicas con soporte de ratón y pantalla táctil.
- **Módulo 6 (Motor Multi-Lienzo en `/experimentos`):** Inicializa todos los lienzos `.multi-exp-canvas` (Gravedad, Ondas y Malla de Brújulas Magnéticas) vinculados a sus controles deslizantes `.lab-slider`.
- **Módulo 7 (Contacto SQLite y Registro PWA):** Envía el formulario `#contact-form` vía `fetch('/api/contact')` y gestiona el registro del Service Worker y el botón `#pwa-install-btn`.

4.3. Motores de Juegos en `static/js/arcade_engines.js`

Cartucho	Características Técnicas Implementadas
01. Buscaminas	5 niveles predefinidos + modo personalizado, primer clic seguro, chording y animación de onda expansiva radial al detonar una mina.
02. Sudoku	Generación aleatoria de tableros válidos, iluminación de fila/columna/subcuadrícula y selector numérico flotante (`#sudoku-popover`).
03. Solitario	Arrastre físico real con Pointer Events Drag & Drop (dedo y ratón), apilamiento en cascada y movimiento rápido con un toque.
04. Tetris	Lienzo HTML5 con cálculo de pieza fantasma (guía de caída), control por gestos táctiles (swipe/tap) y teclado.
05. 2048	Matriz 4x4 con fusión de celdas, detección de fin de partida e interruptor de animaciones ON/OFF persistente.
06. Snake	Lienzo retro con bordes continuos (toroidales), control por deslizamiento táctil o teclado y guardado automático de longitud máxima.

5. Diagnóstico de Errores Encontrados y Solución Aplicada

¿Por qué no funcionaba ningún botón ni los experimentos? (Causa Raíz):
El archivo `static/js/main.js` había sido sobrescrito accidentalmente con el contenido HTML de la plantilla `templates/game_pwa.html` (comenzando con `<!DOCTYPE html>` en lugar de código JavaScript). Al intentar el navegador ejecutar `main.js` como script, lanzaba inmediatamente un error de sintaxis fatal (`Uncaught SyntaxError: Unexpected token '<'` en la línea 1), deteniendo por completo toda la interactividad de la página: botones de navegación, cambio de tema, acordeón de proyectos, demostración de Buscaminas, simulaciones físicas en Canvas y formulario de contacto.

Correcciones y Mejoras Implementadas:

1. **Restauración y mejora de `static/js/main.js`:** Se reconstruyó íntegramente el archivo JavaScript con sus 7 módulos documentados.
2. **Prevención de conflicto en Buscaminas:** Se aisló el motor de demostración de portada en `main.js` (verificando `!document.body.dataset.activeGame`) para evitar colisión de eventos con `arcade_engines.js` en `/arcade/buscaminas/`.
3. **Soporte táctil en Experimentos:** Se agregaron eventos `touchstart` y `touchmove` en los lienzos de física de `index.html` y `experimentos.html` para que respondan tanto al dedo en móviles como al cursor en PC.
4. **Persistencia de Tema y Variables de Entorno:** Se añadió persistencia del modo claro/oscuro en `localStorage`, soporte de `.env` en `app.py` y actualización de caché en los Service Workers (`v5`).

6. Configuración de Variables de Entorno (`.env`) y Entorno Virtual

```
# 1. Crear y activar el entorno virtual (en Linux / Termux)
python3 -m venv .env_venv
source .env_venv/bin/activate

# 2. Instalar dependencias desde requirements.txt
pip install -r requirements.txt

# 3. Ejecutar el servidor Flask
python3 app.py
```